

Stack & Flow (ehem. $\uparrow\downarrow$ = NYT, nützliche Datenflüsse)

Nyt (die nützliche) Flussname im Lied der Grímnismál (Edda):

https://de.wikipedia.org/wiki/Liste_der_Fl%C3%BCsse_im_Lied_Gr%C3%ADmnism%C3%A1l

Stack & Flow ist eine minimalistische, formale Sprache zur Beschreibung aktiver Berechnungen aus laufendem Text heraus. Zum Beispiel kann eine auf dem newtonsche Grundgesetz $F=m \cdot a$ basierende Berechnung wie folgt definiert werden:

§S1 + Für die folgenden Berechnungen im Text wird ein separater Stack S1 angelegt.

Die Beschleunigung auf der Erde beträgt $9,81 \downarrow \text{m/s}^2 \downarrow$. Ein Mensch mit einem Gewicht von $120 \downarrow \text{kg} \downarrow$ wird mit der Kraft $\downarrow F=m \cdot a \downarrow$ angezogen.

Für die Berechnung relevante numerische Werte als auch Strings werden vom Text durch spezielle Präfixe wie F , K und J separiert. Mittels des Operators $\downarrow$ werden diese in einen Stapelspeicher im Hintergrund geschrieben, aus dem dann Funktionen wie $\downarrow F=m \cdot a$ oder $\downarrow$ diese einlesen, verarbeiten und auf den Stapel wieder zurückschreiben. $\downarrow$ liest zum Beispiel den gesamten Stapel aus, und blendet ihn hinter dem Funktionsaufruf in den Text ein.

Grundlagen

Runen als Präfix

Alle für den Parser unterscheidbaren Strukturen erhalten ein Präfix in Form einer nordischen **Rune**.

Die *Runen* werden in keiner heute mehr existierenden Sprache genutzt. Damit sind die Präfixe, durch die Sprachstrukturen kenntlich werden, eindeutig von Textdaten unterscheidbar.

Kommentare +

+ schließt den Rest vom Parsen aus. Damit können nach + beliebige Kommentare notiert werden.

Stapelspeicher & und die Operatoren $\downarrow$ (push) und $\uparrow$ (pop)

Stapelspeicher sind die einzigen, zur Laufzeit veränderliche Speicher in **Stack & Flow**. Operatoren und Programme können aus diesen lesen und ihre Ergebnisse wieder zurückschreiben.

Es können mittels dem Stack- Operator $\&$ beliebig viele Stapel zu Laufzeit definiert und aktiviert werden.

Mittels $\& \text{Stack_Name}$ wird ein Stack angelegt, an den Namen gebunden und aktiviert. Aktiviert bedeutet, dass alle nachfolgenden Stackoperationen für diesen gültig sind.

Die Stackoperation $_Wert \downarrow$ (push) speichert/legt den Wert auf den Stapel. Mittels der Stackoperation $\uparrow$ (pop) kann der Wert wieder vom Stapel genommen werden. $\uparrow \& _name _$ nimmt einen Wert vom Stapel und bindet ihn an den Namen *namen* (siehe unten) .

Wird ein weiterer Stapel mittels $\mathbb{X}$ **Stack_Name_2** angelegt und aktiviert, dann existiert der unter *Stack_Name* zuerst weiter, ist jedoch nicht aktiv. Soll er wieder aktiv werden, dann muss $\mathbb{X}$ **Stack_Name** erneut aufgerufen werden.

```
+ Ein neuer Stack mit dem Namen S1 wird angelegt. Der Stack ist leer []
 $\mathbb{X}$ S1

+ Die drei Werte 1, 2, 3 werden auf dem Stack abgelegt
K1 $\downarrow$  K2 $\downarrow$  K3 $\downarrow$ 

+ Nun hat der Stack den Inhalt Bottom[1][2][3]Top
+ Es wird ein Wert vom Stack genommen, und an den Namen x gebunden
 $\uparrow$  $\mathbb{X}$ x
+ Der Stack S1 hat nun den Inhalt Bottom[1][2]Top

+ Ein neuer Stack mit dem Namen S2 wird angelegt. Der Stack ist leer []
+ Der Stack S1 existiert jedoch weiter
 $\mathbb{X}$ S2

+ In S2 werden zwei weitere Werte abgelegt:
K4 $\downarrow$  K5 $\downarrow$  *x $\downarrow$ 
+ S1 hat den Inhalt Bottom[1][2]Top
+ S2 hat den Inhalt Bottom[4][5][3]Top
```

Literale elementarer Datentypen

Präfixe für die Notation von Zahlenwerten

Eine Gleitpunktzahl wie **3.14** ist eine kulturspezifische Notation (**en-US**).

Um die Notation von Zahlenwert von einer textuellen und kulturspezifischen Präsentation in einer Sprache zu unterscheiden, werden diese in **Stack $\mathbb{X}$ Flow** stets durch ein spezielles *Präfix* explizit gekennzeichnet.

 Zahlen können wie z.B. $\mathbb{R}$ **Zähler*/Nenner** eine listenartige Struktur darstellen, sind aber keine Listen. Die einzelnen Partikel wie im Beispiel **Zähler** und **Nenner** dürfen nur Konstanten sein, wie $\mathbb{R}$ *1/2*, jedoch keine Ausdrücke!

Numerische Datentypen

Die Notationsformen für Zahlenwerte haben Beschränkungen bezüglich der Genauigkeit. Deshalb korrespondieren die Notationsformen auch mit Teilmengen von $\mathbb{Q}$. Diese Teilmengen Werden *Numerische Datentypen* genannt.

Die numerischen Datentypen werden durch Kombination des speziellen Präfixes für eine Notation (z.B. $\mathbb{K}$) mit dem allgemeinen Datentyp- Schalter $\mathbb{T}$ verbunden zum Datentyp Symbol $\mathbb{K}\mathbb{T}$.

$\mathbb{T}$ schaltet allgemein die Evaluierung einer Liste in die Evaluierung einer Typdeklaration um.

$\mathbb{T}$ alleine steht für jeden beliebigen Datentyp.

Basis des Zahlensystems $\mathbb{B}$

Zahlen können über verschiedenen *Basen* dargestellt werden. So kann die **26** dargestellt werden dekadisch mit der Basis **10** als **26**, hexadezimal mit der Basis **16** als **1A**, und binär mit der Basis **2** als **LL0L0**.

Die Basis kann in einen numerischen Typ explizit definiert werden mit dem Präfix $\mathbb{B}$

```
+ Basis 2
 $\mathbb{B}2$ 

+ Basis 10- kann in der Regel entfallen, da Default
 $\mathbb{B}10$ 

+ Basis 16
 $\mathbb{B}16$ 
```

Kardinalzahlen $\mathbb{K}$

$\mathbb{K}$ ist das Präfix für ganze Zahlen:

```
 $\mathbb{K} 1$             $\iff 1$ 
 $\mathbb{K} -123$          $\iff -123$ 
 $\mathbb{K} \mathbb{B}16 \text{ AFD}$     $\iff \text{nat. Zahl zur Basis 16 (hex)}$ 
 $\mathbb{K} \mathbb{B}2 \text{ L00LLL}$   $\iff \text{nat. Zahl zur Basis 2 (dual)}$ 
 $\mathbb{K} \mathbb{M}$             $\iff + \text{ Unendlich}$ 
 $\mathbb{K} -\mathbb{M}$            $\iff - \text{ Unendlich}$ 
```

$\mathbb{K}\mathbb{T}$ ist der Datentyp für Kardinalzahlen.

Gebrochen Rationale Zahlen $\mathbb{R}$

$\mathbb{R}$ ist das Präfix für gebrochen rationale Zahlen. Diese bestehen aus einem *Nenner* und einem *Zähler*, getrennt durch ein /. Die Rune $\mathbb{X}$ (Gebo) präfixt den Exponenten. Per default ist die *Basis 10*, auch für den *Exponenten*. Mittels $\mathbb{B}$ kann eine abweichende *Basis* vereinbart werden.

1. $\mathbb{R}$ **Zähler** hier ist der Nenner stets 1
2. $\mathbb{R}$ **Zähler** / **Nenner**
3. $\mathbb{R}$ **Ganzzahlig** **Zähler** / **Nenner**
4. $\mathbb{R}$ **Ganzzahlig** **Zähler** / **Nenner** $\mathbb{X}$ **Exponent**
5. $\mathbb{R} \mathbb{B}$ **Basis** **Ganzzahlig** **Zähler** / **Nenner** $\mathbb{X}$ **Exponent**

Beispiele:

```
 $\mathbb{R} 2$             $\iff 2/1 = 2.0$ 
 $\mathbb{R} 1/2$          $\iff 1/2 = 0.5$ 
 $\mathbb{R} 1 \text{ 2/3}$       $\iff 1 \text{ 2/3} = 1.666$ 
 $\mathbb{R} -4/16$        $\iff -4/16 = -0.25$ 
```

```
R B2 -L00/L0000 ⇔ -4/16 = -0.25 im binärsystem
R B2 -L/L000 XLL ⇔ -1 = -1/8 * 2^3
```

Die rationalen Zahlen können z.B. als Zoll- Maße genutzt werden

R ist der Datentyp für gebrochen rationale Zahlen.

Gleitpunktzahlen **F**

F ist das Präfix für rationale Zahlen in der Gleitpunkt- Darstellung. Vor- und Nachkomma- Stellen werden durch , getrennt. Einen Exponenten zu Basis 10

```
F 3          ⇔ 3.0
F 3,14       ⇔ 3.14
F -2,72      ⇔ -2.72
F -2,72X3    ⇔ -2.72e3 = -2720
F B2 -L00 L0000 ⇔ -4,5 (binär)
```

F ist der Datentyp für Gleitpunkt- Zahlen.

Die Datentypen **R** und **F** sind kompatibel bzw. austauschbar: Ein **R** kann an ein **F** zugewiesen werden und umgekehrt.

Komplexe Zahlen **ℂ**

Komplexe Zahlen können als Punkte der gausschen Zahlenebene betrachtet werden. Dies legt die Darstellung als Wertepaar **(re, im)** mit re= Realteil, und im= Imaginärteil nahe.

Alternativ kann man komplexe Zahlen als die algebraische Kombination **re + iim** von reellen und imaginären Zahlen betrachten. Dabei ist **re** ∈ ℝ und **iim** ∈ ℐ. Der Imaginärteil erhält dabei das mathematische Schreibschrift **i** (U + 1D4BE) als Präfix.

Boolsche Werte **B**

B ist das Präfix für boolsche Werte. Die beiden möglichen boolschen Werte werden durch die Namen **true** und **false** ausgedrückt:

```
B true  ⇔ True
B false ⇔ False
```

B ist der Datentyp für boolsche Werte.

Strings **J**

Strings sind Listen aus beliebigen Zeichen. Sie können auch Leerzeichen enthalten.

Strings, die keine Leerzeichen enthalten, können direkt notiert werden.

```
+ geschlossener String, enthält keine Leerzeichen
Hallo

+ geschlossener Strings, die einzelne Hierarchieebenen benennen
℘ All Galaxieen Andromeda ℑ
```

Enthalten *Strings* Leerzeichen, dann müssen sie in ein **S-Array**: `ℑ ... ℑ` gesetzt werden. `ℑ` ist das Präfix für String- Listen.

Die Leerzeichen sind innerhalb eines String- Array geschützt.

Sehr Lange Strings können mittels `ℑ` auf mehrere Zeilen umgebrochen werden.

```
+ String aus mehreren Wörtern. Die Leerzeichen sind geschützt
ℑHallo   Weltℑ

+ Komplexe Texte als String, umgebrochen auf mehrere Zeilen mittels ℑ
ℑ Mit Strings können auch **MarkDown** formatierte Texte geschrieben werden.ℑ
So wird *Text* und *Logik* vollständig vermischt.ℑ
```

`ℑℑ` ist der Datentyp für Strings.

Stringinterpolation

Werden in einem String Namensreferenzen eingesetzt, die beim Abruf des Strings evaluiert werden, dann liegt eine Stringinterpolation vor.

Sei `ℑattrib schöne` eine Namensbindung. Dann kann eine Stringinterpolation wie folgt definiert werden:

```
ℑ Hallo ℑattrib Welt ℑ
```

Diese wird dann evaluiert zu:

```
ℑ Hallo schöne Welt ℑ
```

Hierarchieen ℘

℘ (runic Fehu) ist das Präfix eines Pfades in einer Hierarchie. Der Pfad muß durch ein Listenendsymbol ℑ (runic Q) abgeschlossen werden.

`℘ K23 K10 K15 ℑ` ⇔ Kann z.B. eine Versionsnummer mit den drei Hierarchieebnen *Hauptversion*, *Nebenversion*, *Buildnummer* darstellen. Oder die Uhrzeit **23:10:15**. Oder das Datum **15.10.2023**.

`℘ millingMachine circelMilling millDiscℑ` ⇔ Pfad in einem Namensraum

`℘ℑ` ist der Datentyp für Hierarchieen.

Informationen darstellen durch Binden von Werten an Namen mittels `ℑ` Operator

Informationen bestimmen den Ausgang von Entscheidungen. Entscheidungen manifestieren sich durch die Zuordnung/Belegung/Bindung von Werten an Systemparametern.

Die Bindung eines Wertes an einen Systemparameter wird **Attribut** genannt.

Mittels dem **Bind** Operator $\mathfrak{X}$ (runic Othalan) kann in **Stack $\mathfrak{X}$ Flow** ein Wert an einen Namen gebunden, und somit ein Attribute gebildet werden. Über den Namen ist der Wert dann referenzier- und abgerufenbar.

Der Namen muss innerhalb seines *Namensraumes* (siehe unten) eindeutig sein. Ein typischer Namensraum ist die **Stack $\mathfrak{X}$ Flow** Datei.

Attribute bzw Namensbindungen sind wie folgt aufgebaut: $\mathfrak{X}$ <Name als String> <Wert>

Beispiele:

```
+ Konstante PI definieren
 $\mathfrak{X}$ PI  $\mathfrak{F}$ 3,14

+ Liste der ersten fünf Primzahlen an einen Namen binden
 $\mathfrak{X}$ ersteFünfPrimzahlen  $\mathfrak{N}$ K2 K3 K5 K7 K11  $\mathfrak{I}$ 
```

Die Bindung eines Namens an einen Wert kann auch als **Attribut Wertepaar** betrachtet werden!

Zu bindende Werte könne mit $\uparrow$ vom Stapel gelesen werden:

```
Die Erdbeschleunigung beträgt  $\mathfrak{F}$ 9,81 $\downarrow$   $\mathfrak{J}$ m/s2 $\downarrow$ .
 $\uparrow$  $\mathfrak{X}$ g          + Wert für Erdbeschleunigung vom Stack lesen und an Name g binden
 $\uparrow$  $\mathfrak{X}$ gunit      + Einheit für Erdbeschleunigung vom Stack lesen und an Name gunit
binden
```

Naming ID's $\mathfrak{H}$

$\mathfrak{H}$ ist das Präfix für eine *NamingID*. Eine *NamingID* ist ein global eindeutiger Name in Form einer **64bit GUID**.

An eine global eindeutige *NamingID* kann wie an einen lokalen *Namen* ein Wert gebunden werden in der Form $\mathfrak{X}\mathfrak{H}$ K $\mathfrak{B}$ 16 <Hex- Wert Naming ID> _Attribut-Wert_.

Beispiele:

```
+ An die global gültige Naming ID 0x7ABC123 wird der Wert 3,1427 gebunden.
 $\mathfrak{X}\mathfrak{H}$  K  $\mathfrak{B}$ 16 7ABC123  $\mathfrak{F}$ 3,1427
```

$\mathfrak{H}\mathfrak{T}$ ist der Datentyp für Namensreferenzen.

Namen in einen Wert auflösen mittels $\mathfrak{K}$ (lor)

Wurde an einen Namen ein Wert gebunden, dann kann überall, wo normalerweise der Wert eingesetzt wird, der Name eingesetzt werden. Dazu ist dem Namen das runic lor $\star$ voranzusetzen:

```
+ Konstante PI definieren
 $\star$ PI f3,14

+ Den Wert von **PI** an den synonymen Namen **pie** binden
 $\star$ pie  $\star$ PI

+ Den Wert der globalen mit Naming ID definierten Konstante **PI** an den
synonymen Namen **piGlob** binden
 $\star$ piGlob  $\star$ H K B16 7ABC123I
```

Namensraum- Strukturen $\star \dots \mathcal{P} \dots \mathcal{N}$

Eine Menge von *Bind* Operationen können in Listen $\mathcal{P} \dots \mathcal{N}$ zusammengefasst werden. Innerhalb einer solchen Liste darf ein bestimmter Name stets nur einmal an einen Wert gebunden werden. Diese Listen werden **Namensraumstruktur**, oder kurz **Struktur** genannt.

```
+ Beschreibung einer Punktkoordinate durch eine Namensraumstruktur
 $\mathcal{P}$   $\star$ x f2,72  $\star$ y f3,14  $\mathcal{N}$ 
```

Die Struktur kann selber als Wert mittels Bind an einen Namen gebunden werden. So entsteht ein **Namensraum**

```
+ Namensraum mathematischer Konstanten
 $\star$ MathConst
 $\mathcal{P}$ 
     $\star$ PI f3,14
     $\star$ e f2,72
 $\mathcal{N}$ 

+ Namensraum, der einen Punkt darstellt
 $\star$ Punkt1
 $\mathcal{P}$ 
     $\star$ x f2,72
     $\star$ y f3,14
 $\mathcal{N}$ 
```

Hierarchische Namensraum Referenzen $\mathcal{H}\mathcal{P} \dots \mathcal{N}$

Die Namen in einem Namensraum sind außerhalb dieses nicht mehr eindeutig. Eine eindeutige Addressierung der Attribute aus einer Position im Code außerhalb eines Namensraumes werden *Hierarchische Namensraum Referenzen* benötigt. Diese haben folgenden Aufbau: $\mathcal{H}\mathcal{P}$ <Name 1. Level> <Name 2. Level> ... <Name N. Level> $\mathcal{N}$.

Für den Zugriff auf den Werte ist er hierarchischen Referenz der *for* Operator * voranzustellen: *H P <Name 1. Level> <Name 2. Level> ... <Name N. Level> P.

Beispiel

```
+ Organisation einer mathematischen Bibliothek
xMath
P
  xConst
  P
    xPI f3,14
    xe f2,72
  P
P

+ Zugriff auf PI
*H PMath Const PI P
```

Namensräume auf Basis globaler Naming- IDs

Um abstrakte Naming- IDs besser zu handhaben, können sie an lesbare Namen mittels x gebunden, und diese lesbaren Namen in Namensraumstrukturen organisiert werden:

```
+ Organisation einer mathematischen Bibliothek, 2
xMath
P
  xBasicFunctions
  P
    + Naming- IDs der math. Grundrechenarten werden an lokale Namen gebunden
    xadd *H K B16 ADDADD
    xsub *H K B16 DE2323
  P
P

+ Zugriff auf add
*H PMath BasicFunctions add P

+ Hier wird über den hierarchischen Namen die Funktion aufgerufen
^H PMath BasicFunctions add P K1 K2
+ xsum ist nur innerhalb des Siegel - Zweiges sichtbar
h ^xsum ^print J *sum ist die Summe aus 1 und 2 P
```

Arrays P

Arrays sind Listen von Werten. Die Werte können primitiv oder komplex sein.

P (runic Y) ist das Präfix, welches die Liste eines Arrays eröffnet. P beendet die Liste.


```

+ Array mit den ersten fünf Primzahlen
N K2 K3 K5 K7 K11 ↵

+ Array mit zwei Koordinaten
N
  P x F2,72 y F3,14 ↵
  P x F5,3   y F1,71 ↵
↵

+ Array aus Daten verschiedener Typen
N
  P x K2 y K3 ↵
  K13
  J Summe aus a² und b² ↵
↵

```

NT Hred Hgreen Hblue ↵ steht für einen Aufzählungstyp/Set: Eingesetzt werden dürfen nur die im Array aufgelistete Werte.

Zugriff auf Array Elemente

Array sind wie alle Werte unveränderlich (immutable): sie können nur gelesen, jedoch nicht verändert werden!

Auf einzelne Elemente eines Arrays kann mittels Indexzugriffs- Operator **[index]** lesend zugegriffen werden. Dieser hat als Parameter den **0** basierte Index. Erw wird direkt auf Array angewendet:

```

+ An den Namen **dritterEintrag** ist nun der Wert K5 gebunden.
xdritterEintrag N K2 K3 K5 K7 K11 ↵[2]

+ An den Namen **eineListe** wird ein Array gebunden
xeineListe N K2 K3 K5 K7 K11 ↵

+ Der 3. Eintrag im Array wird ausgelesen und auf den Stapel gestellt.
*eineListe[2]↓

```

Im letzten Beispiel wird die Priorität der Operatoren deutlich: höchste Priorität hat *, dann kommt [], und schließlich ↓.

Einbetten von Array in Array mittels Expand X Operator

Mittels des Expand- Operator X kann der Inhalt eines Array in ein anderes eingebettet werden

```

xsubArray N K2 K3 ↵

+ *notExpandedArray ist N K1 N K2 K3 ↵ K4↵
xnotExpandedArray
N
  K1

```

```

    *subArray
    K4
  1

+ *res1 hat den Wert 1 K2 K3 1 (Array)
&res1 *notExpandedArray[2]

+ *expandedArray ist 1 K1 K2 K3 K4 1
&expandedArray
1
    K1
    X*subArray
    K4
  1

+ *res21 hat den Wert K3 (einzelnener, ganzzahliger Wert)
&res2 *expandedArray[2]

```

Einbetten von Arrays auf den aktuell aktiven Stack mittels Expand X Operator

Mittels des Expand- Operator X können alle Elemente eines Array hintereinander auf den Stapel kopiert werden:

```

&myArray 1 K1 K2 1

+ Array auf den Stapel kopieren
*myArray↓

+ Der Stapel hat nun die Belegung:
+ [1 K1 K2 1] Top

+ Jetzt werden anstatt des Array selbst die einzelnen Werte des Array auf den
Stapel kopiert
*myArrayX↓

+ Der Stapel hat nun die Belegung:
+ [K2      ] Top
+ [K1      ]
+ [1 K1 K2 1] Bottom

```

EVA: Eingabe, Datenverarbeitung, Ausgabe

```

      +-----+
E1 -->|         |
:     | Verarbeitung | --> Ausgabe
En -->|         |
      +-----+

```

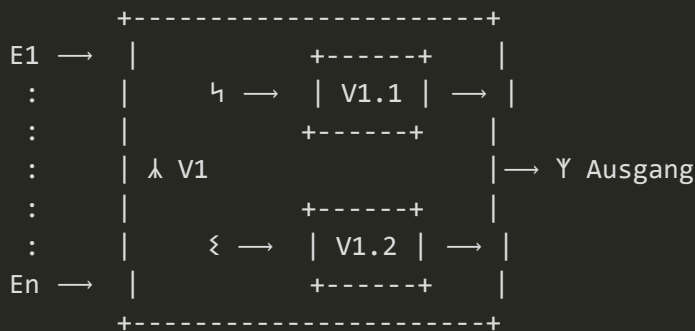
Dieses uralte Prinzip der elektronischen Datenverarbeitung wird wieder in den Fokus gestellt. Die Verarbeitung von Daten erfolgt durch einzelne, benannte *Verarbeitungsstufen* (kurz *Stufe*). Jede Verarbeitungsstufe wird mit der Rune **CALC** λ eingeleitet, ihr folgt der Name der Verarbeitungsstufe, die Eingangsparameter, die Verarbeitungszweige und schließlich der Ausgang, der mit der Rune **EOLHX** Υ gekennzeichnet wird.

```
+ Syntaktischer Aufbau einer Verarbeitungsstufe
 $\lambda$  _NameVStufe_ _E1_ ... _En_
 $\hookleftarrow$  _Nachfolgende_Verarbeitungsfunktion_im_SIGEL_Zweig_
 $\xi$  _Nachfolgende_Verarbeitungsfunktion_im_SOWILO_Zweig_
 $\Upsilon$  _Abschluss_oder_Nachfolge_Funktion_am_Ausgang_
```

In **Stack Flow** kann die Verarbeitung in einer Stufe stets in zwei alternative Pfade erfolgen. Damit wird das grundlegende Prinzip der Verzweigung eingeführt.

- $\hookleftarrow$: SIEGEL Zweig
- ξ : SOWILO Zweig

Am Ende müssen aber beide Pfade wieder am Ausgang zu einem Pfad zusammengeführt werden. Damit können komplexe, jedoch strukturierte Datenflussgraphen konstruiert werden:



Eingangswerte/Paramter

Jede Stufe kann parametrierbar werden. Die Parameter (oder Eingangswerte) werden auf dem Stapelspeicher bereitgestellt. Der Stapelspeicher kann unmittelbar nach dem Stufenamen mittel $\downarrow$ (push) Operatoren vor Aufruf der Stufe mit den benötigten Parametern befüllt werden.

Eine einfache Verarbeitungsstufe, die dieses Prinzip direkt auszunutzt, ist die **push** Stufe. Sie legt alle Eingangsparameter unverändert auf dem Stapel des Laufzeissystems ab:

```

Inhalt Stapelspeicher
--++--++--
 $\lambda$  push  a ... z  a | b | c |
--++--++--
 $\hookleftarrow$   $\hookleftarrow$   $\downarrow$ 
 $\xi$   $\downarrow$ 
 $\Upsilon$ 

```

Annahmen zum Stapelspeicher definieren

Da jede Stufe ihre Parameter vom Stapel liest, muss sichergestellt werden, dass auch alle benötigten Parameter auf dem Stapel für die Stufe bereitstehen. Die Prüfung des Stapelspeichers erfolgt durch die Stufe zur Laufzeit. NYT stellt zudem eine generische Implementierung für solche Prüfungen bereit durch

Musterbelegungen:

Def Musterbelegung: ist eine Liste von Typnamen nach der INGWAZ Rune: $\diamond \Upsilon_1 \dots \Upsilon_n$. Der erste Typname Υ_1 bezeichnet dabei den Datentyp des ersten Wertes auf dem Stapelspeicher, der zweite Υ_2 den des zweiten Wertes auf dem Stapelspeicher usw..

Die **Musterbelegung** kann an die Parameterliste einer Stufe angehängen werden, und definiert eine Annahme über die Belegung des Stapelspeichers vor dem Einkellern der Parameter einer Stufe:

$p_1^\downarrow \dots p_n^\downarrow$	$\lambda \text{stufenName} \diamond \Upsilon_1 \dots \Upsilon_m$	Υ
$\backslash \text{---+---} /$	$\backslash \text{---+---} /$	
Einzukellernde	Annahme über die bereits auf	
Parameter	dem Stapel liegenden Parameter	

Wenn eine **Musterbelegung** nicht zutrifft, dann wird eine Fehlermeldung erzeugt und auf dem Stapel abgelegt. Anschließend wird im ξ (**Sowilo**) Zweig der Stufe fortgesetzt.

 **Achtung:** Die Musterbelegung scheint einer formalen Parameterliste einer Prozedur in einer Programmiersprache wie **C#** zu entsprechen. Jedoch handelt es sich hier um ein automatisiertes Prüfverfahren für die Stapelspeicherbelegung zur Laufzeit (keine Prüfung zur Entwurfszeit via Compiler!), die beim konkreten Start der Stufe stattfindet. Es kann deshalb für verschiedene Stufenstarts auch verschiedene Musterbelegungen geben:

```
K77↓ K88↓

+ Hier wird eine Musterbelegung von zwei Kardinalzahlen auf dem Stapelspeicher
angenommen.
λadd ◊ K↑ K↑
ξ λprint↑
↑
```

Zweige

Eine Methode verarbeitet die übergebenen Parameter. Danach gibt es drei Möglichkeiten der Programmfortsetzung:

1. Es wird im η (**Siegel**) **Zweig** fortgesetzt
2. Es wird im ξ (**Sowilo**) **Zweig** fortgesetzt

3. Es wird sofort zum Stufenausgang Υ (**Eolhx**) gesprungen und diese damit formal beendet

In den Fällen 1 und 2 wird nach Durchlauf der Zweige ebenfalls am Ausgang Y abgeschlossen.

Beispiele:

```
+ Wurzel aus einer Zahl a ziehen
a ↵ ↵SQRT
    ↳ _op_auf_va_           + Hier wird die √ von a bereitgestellt
    ↳ _Fehlerbehandlung_    + z.B. im Fall a < 0
    ↳ _Abschlussfunktion_   + Hier wird der Stapelspeicher Nach Ausführung von ↳
oder ↳ bereitgestellt
```

Bereitstellung der Ergebnisse in den Zweigen

↓ (push) und ↑ (pop)

```

a↓ b↓ c↓
      --+---+---+
λm      a | b | c| -----+-----+   + Ablage der Parameter auf dem
Stapelspeicher
      --+---+---+   |   |
      -----+---+   |   |
+---  4 λsY   m(a, b, c) | <-+   |   + Ergebnis der Methode s im 4 Zweig
bereitstellen
|           -----+   |
|           -----+   |
| +--  3 λeY   m(a, b, c) | <-----+   + Ergebnis der Methode e im 3 Zweig
bereitstellen
| |           -----+
| |           -----+
| |           -----+
+--+> Y       s(m(a, b, c)) oder e(m(a, b, c)) |   + Ergebnis vom 4 oder 3 Zweig
bereitstellen
|           -----+

```

Beispiel: Berechnen der Quadratwurzel

```
Ja²=1↓
λ input
↳ ↑xaa
+ Ende von Input
Υ

↳ Es wird nun die Wurzel aus *aa gezogen 1↓ λprintΥ

+ Start Wurzel ziehen (Inhalt von xaa wird auf den Stapel gelegt)
*aa↓
```

```

λ sqrt

+ Weiterleiten des Ergebnisses an die Print- Methode. Achtung: Im AusgabeString
findet
+ String- Interpolation statt.
↳ Jv*aa= 1↓ K2↓ λprint ◊ F↑ J↑ K↑Y

+ Weiterleiten im Fehlerfall an die Print- Methode. Achtung: Im AusgabeString
findet
+ String- Interpolation statt.
ξ Jv*aa ist konnte nicht ermittelt werden. Ursache: 1↓ K2↓ λprint ◊ J↑ J↑ K↑Y

+ Hier werden die Ausführungspfade wieder zusammengeführt
Y
JProgramm ✓ beendet.1↓ K1↓ λprintY

```

Hintereinanderschalten von Stufen in Sequenzen

Verarbeitungsstufen können direkt hintereinander ausgeführt werden: $\lambda V1Y \lambda V2Y \dots \lambda VnY$. Die Ausgaben der ersten landen dabei auf dem Stack, von dem sie die zweite Verarbeitungsstufe einlesen und weiterverarbeiten kann usw.

```

+ Input liest einen Wert von der Tastatur ein und legt ihn auf den Stack
λinput J gib eine ganze Zahl z ein. Der absolute Betrag |z| wird ermittelt ! 1 Y

+ Vergleichsoperator 0 > eingabe
λa_gt_b K0 ◊ K↑Y

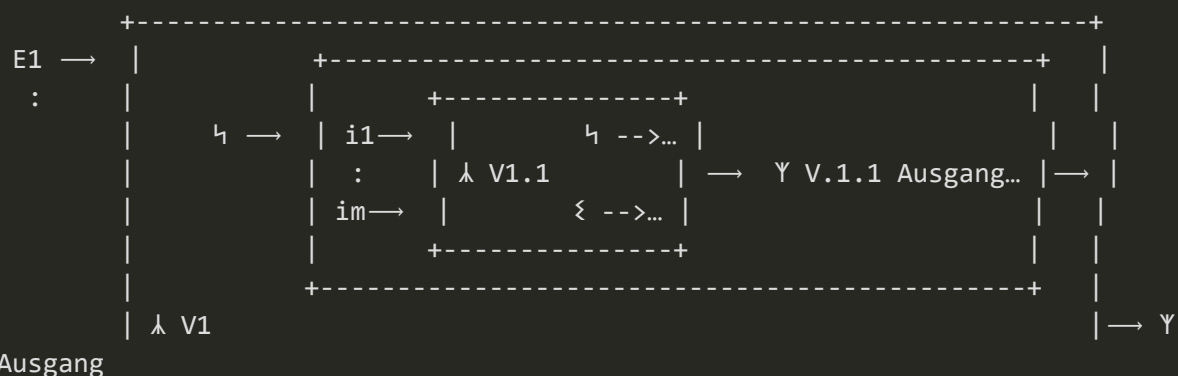
+ Auf dem Stack liegt das Ergebnis von 0 > eingabe
λifElse ◊ B↑ ↳ λmul K-1 ◊ K↑ Y + Wenn eingabe < 0 ist, dann mit -1 multiplizieren

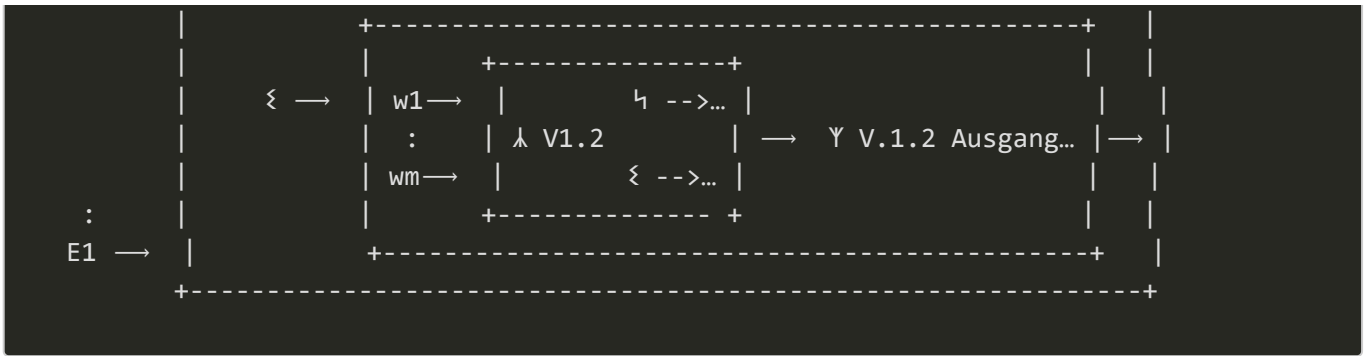
+ Print liest die nächsten beiden Werte vom Stack, und gibt sie aus.
λprint K2 J Der absolute Betrag |z| = 1 ◊ K↑ Y

```

Verschachtelung von Stufen

In den $\hookrightarrow$, ξ und Y Zweig kann der bereitgestellte Inhalt des Stapelspeichers jeweils durch weitere Verarbeitungsstufen verarbeitet werden:





Durch Fortsetzen dieses Prinzips können tief verschachtelte Strukturen entstehen.

```
+ Input liest einen Wert von der Tastatur ein und legt ihn auf den Stack
λinput J gib eine ganze Zahl z ein. Der absolute Betrag |z| wird ermittelt ! ¶

+ Falls keine Eingabe erfolgte (Abbruch), weiter im Sowilo Zweig
Σ λprint K1 J Die Eingabe wurde abgebrochen ¶ ¶

+ Eine Eingabe wurde erfolgreich durchgeführt: weiter im Siegel Zweig
¶ λa_gt_b K0 + Vergleichsoperator 0 > eingabe
Σ print J Fehler: Der Wert auf dem Stack ist keine Zahl und kann nicht
verglichen werden !¶ ¶
¶ + Auf dem Stack liegt das Ergebnis von 0 > eingabe
λifElse

+ Wenn eingabe < 0 ist, dann mit -1 multiplizieren
¶ λmul K-1¶

+ Wenn eingabe >= 0 ist, dann mit 1 multiplizieren
Σ λmul K1¶
¶
¶
¶ + Hier kann nun der absolute Betrag auf dem Stapel

λprint K2 J Der absolute Betrag |z| = ¶
Σ print K1 J Der Stapel ist leer ¶
¶
```

Abrufen des Ergebnis- Stapelspeichers als Array ¶

Der gesamte Stapelspeicher kann in einem ¶, Σ und ¶ Zweige als das spezielle Array ¶ abgegriffen werden. Mittels ¶ Parallel- Zugriffoperator können einzelne Elemente herausgegriffen und gezielt weiterbearbeitet werden. Zur Laufzeit wird jedes herausgegriffenen Element in einem eigenen *Laufzeittask* bearbeitet- der ¶ ist damit das primäre Instrument zur Parallel- Programmierung. ¶ hat folgende Signatur:

```
¶ _Index1_ [_index2 [ ... [index n]]]
¶ _meth_für_Zweig1_ + Methode, die auf den Wert mit Index 1 aus ¶ angewendet
wird
¶ _meth_für_Zweig2_ + Methode, die auf den Wert mit Index 2 aus ¶ angewendet
```

```
wird
:
  ↳ _meth_für_ZweigN_   + Methode, die auf den Wert mit Index N aus  $\mathfrak{N}$  angewendet
wird
  ↳ _meth_für_einen_outOfRange_Fehler_
  ↳ _Folgefunktion_
```

Der Wert zu jedem Index wird an einen korrespondierenden $\hookrightarrow$ Zweig geleitet, und kann dort mit einer Folge-Methode weiterbearbeitet werden.

Sollte ein Index außerhalb des Stapelspeicher- Array $\mathfrak{N}$ liegen, dann wird kein $\hookrightarrow$ Zweig betreten, sondern nur der ξ Zweig. In diesem kann eine Fehlerbehandlung stattfinden.

Υ wird in jedem Fall am Ende durchlaufen. Hier kann eine Folgefunktion gestartet werden. Im Kontext der parallelen *Laufzeittasks* stellt hier Υ einen *Join* dar.

Benennen von Ergebnissen einer Stufe

Alternativ zum Abruf und Weiterverarbeitung der Ergebnisse mit $\mathfrak{N}\uparrow$ können die Einträge am Stufenausgang auch aus dem Stapelspeicher gelesen und benannt werden mit $\mathfrak{X}$:

```

      --+---+---+
 $\lambda$  m a b c   a | b | c | -----+-----+   + Ablage der Parameter auf dem
Stapelspeicher
      --+---+---+           |           |
      -----+           |           |
+---  $\hookrightarrow$  s      m(a, b, c) | <-+           |   + Ergebnis der Methode im  $\hookrightarrow$  Zweig
bereitstellen
|           -----+           |
|           -----+           |
| +---  $\xi$  e      m(a, b, c) | <-----+   + Ergebnis der Methode im  $\xi$  Zweig
bereitstellen
| |           -----+
| |
| |           -----+
+-->  $\Upsilon$   $\mathfrak{X}$ res  s(m(a, b, c)) oder e(m(a, b, c)) |   + Ergebnis aus  $\hookrightarrow$  oder  $\xi$  an
Namen res binden
      -----+
```

Das benannte Ergebnis kann dann im Folgenden weiterverwendet werden:

```
 $\mathfrak{X}$ a K2
 $\mathfrak{X}$ b K3

 $\lambda$ squ *a
 $\Upsilon$   $\mathfrak{X}$ aa

 $\lambda$ squ *b
 $\Upsilon$   $\mathfrak{X}$ bb
```



```

λadd *aa *bb
+ Weiterleiten des Ergebnisses an die Print- Methode. Achtung: Im AusgabeString
  findet
+ String- Interpolation statt.
Υ print ⌈ *a² + *b² = ⌋

```

Benennungen innerhalb von λ und ξ Zweig

Eine Benennung inner halb eines λ und ξ Zweiges ist nur lokal innerhalb dieses sichtbar.

```

⊗aa K2

λsquRoot *aa
+ ⓧa_lok ist nur innerhalb des Siegel - Zweiges sichtbar
λ ⓧa_lok λprint ⌈ *a_lok ist die Wurzel aus *aa ⌋
ξ λprint ⌈ *aa ist keine reele Quadratzahl! ⌋ Υ
  λpush K0 Υ
+ ⓧa_glob ist für den gesamten Kontext sichtbar, innerhalb dessen squRoot
  aufgerufen wurde
Υ ⓧa_glob

```

Benennen von Datenflussgraphen: Module M ... M

Komplett ausprogrammierte Datenflussgraphen können zwecks Wiederverwendung in Modul- Deklarationen eingeschlossen werden.

Eine Moduldeklaration ist ein Block, der zwischen M und M eingeschlossen wird. Dem M folgt der Name des Moduls:

```

M moduleName
+ Hier wird der Wiederverwendende Datenflussgraph definiert.
M

```

Das Modul kann dann später wie eine elementare Datenverarbeitungsstufe mit den Zweigen λ und ξ verwendet werden. Wann λ und wann ξ aufgerufen werden, kann innerhalb des Moduls mit $\lambda\uparrow$ und $\xi\uparrow$ definiert werden:

```

M divKardinal
+ Hier wird der Wiederverwendende Datenflussgraph definiert.
λpop ◊ K↑ K↑
ξ λpush ⌈ Err divKardinal ⌋
  ξ↑
Υ ⓧNom ⓧDenom

```

```
λifElse ◊ KΥ
```

```
Υ
```

```
⌘
```

Von der Laufzeitumgebung bereitgestellte Stufen

Die Laufzeitumgebung hat bereits eine Reihe von Stufen vordefiniert und implementiert.

Programende

Diese Stufe beendet in jedem Fall das Programm.

```
λfinΥ
```

Fehlerlog

Diese Stufe gibt die als Eingang E1 vorliegende Meldung und den aktuellen Stapelspeicherinhalt in einem Fehlerlog aus. Nach Ausführung dieser Stufe ist der Stapelspeicher in genau dem gleichen Zustand wie vor der Stufe.

```
λlogErr J FEHLERMELDUNG 1Υ
```

Infolog

Diese Stufe gibt die als Eingang E1 vorliegende Meldung und den aktuellen Stapelspeicherinhalt in einem Info- Log aus. Nach Ausführung dieser Stufe ist der Stapelspeicher in genau dem gleichen Zustand wie vor der Stufe.

```
λlogInf J FEHLERMELDUNG 1Υ
```

Stapelspeicher mit Werten füllen

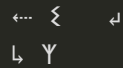
Mit dieser Stufe kann eine Liste von Werten auf den Stapelspeicher gelegt werden. Eine weitere Bearbeitung der Werte auf dem Stapelspeicher findet nicht statt.

Nachfolgende Stufen können die unveränderten Werte auf dem Stapelspeicher dann weiterverarbeiten.

```

                Inhalt Stapelspeicher
                --+---+---+
λ push  a ... z  a |...| z|
                --+---+---+
                ←...  ↳  ↳

```



Alternative Verarbeitung ifElse

Diese Stufe nutzt die Struktur der alternativen Ausführungspfade $\mathcal{H}$ und $\mathcal{Z}$ aus, um eine elementare Verzweigung in Abhängigkeit eines booleschen Wertes zu implementieren.

Der Eingangsparameter von **ifElse** muss ein boolescher Wert sein. Ist er True, dann wird der $\mathcal{H}$ Zweig, sonst der $\mathcal{Z}$ ausgeführt. Am Ende wird wieder im Ψ Zweig zusammengeführt.

```
λifElse _boolscherEingang_
  ℋ _Folgestufe_if_TRUE_
  ℤ _Folgestufe_if_FALSE_
  Ψ _FolgeStufe_von_ifElse_
```

Datenstrom- orientierte Ausgabe

Es können Ausgaben in Dateien erfolgen. Dazu sind diese in einer Stufe zuerst als Datenströme zu öffnen, und dann können Teile des Stapelspeichers in diese ausgegeben werden.

```
λout _name_output_Stream_ _E1_ ... _En_
  ℤ _Verarbeitungsstufe_falls_Ausgabe_scheitert_
  Ψ
```

Interaktives Parsen von LLP

Es ist ein Editor für LLP zu implementieren, der den Benutzer aktiv bei der Eingabe unterstützt.

Nach jedem vollständig eingegeben Wort kann z.B. der Parser gestartet werden.

Z.B. folgende Sitzung:

⌘ _

Der Parser erkennt das Prefix für semantische Beziehungen. Nun kann die Definition oder die Abfrage einer semantischen Beziehung folgen.

⌘ ↑ _

[#1] ↑ - semantische Beziehung abfragen [#2] ℋ - semantische Beziehung definieren: _NID_Referring

Nachdem [#1] gewählt wurde, ist nun eine der möglichen semantischen Beziehungen auszuwählen

⌘ ↑ $\mathbb{H}$ isInstanceOf

[#1] $\mathbb{H}$ isPartOfSemContext [#2] $\mathbb{H}$ isInstanceOf [#3] $\mathbb{H}$ isPartOf [#4] $\mathbb{H}$ isSubTermOf [#5] $\mathbb{H}$ isSubNamespace

Nachdem [#2] gewählt wurde, gibt es eine große Auswahl von Namenscontainern, die Klassennamen von Klassen darstellen, mit denen andere Namenscontainer in der Beziehung **isInstanceOf** stehen können. Hier gibt es verschiedene Strategien, um den gesuchten Namenscontainer des Klassennamens zu finden:

1. Über den Namensraum- Pfad zur Naming ID des gesuchten Namenscontainers navigieren.

1. Es werden nur Namensraum- Pfade unterstützt, die Klassennamen enthalten

2. Es werden alle Namensraumpfade unterstützt. In einem Namensraum werden nur Namenscontainer von Klassennamen angezeigt.

2. Über ein Autocomplete- Textcontrol, das nur die CNT's von Klassennamen unterstützt, den CNT auswählen lassen.

Sei Variante 2 der Standard- Modus. In Variante 1 kann bei Bedarf umgeschaltet werden:

⌘ ↑ $\mathbb{H}$ isInstanceOf $\mathbb{H}$ milProgram

[#1] Select Naming- Containear with class name via Namespace